

UNIVERSIDAD PERUANA UNIÓN

Desarrollo de aplicaciones móviles multiplataforma

INFORME DE IMPLEMENTACIÓN

BiblioMobil

Sistema móvil multiplataforma para la gestión de una biblioteca universitaria

Estudiante	Guillen Mendoza Alexander
Docente	David Reyna
Rama de entrega	feature/clean-mvvm
Fecha	15 de septiembre de 2026

Proyecto independiente desarrollado con Kotlin Multiplatform, Compose Multiplatform, Clean Architecture, MVVM, StateFlow y Koin.

1. Resumen ejecutivo

BiblioMobil es una aplicación móvil independiente orientada a bibliotecarios. Permite registrar y consultar libros y lectores, conserva preparado el módulo de préstamos y comparte la interfaz y la lógica entre Android e iOS. La solución cumple la regla de dependencia de Clean Architecture: presentación invoca casos de uso, dominio declara los modelos y puertos, y datos implementa esos puertos sin que el dominio conozca la tecnología de persistencia.

1.1 Resultado de la entrega

Aspecto	Resultado
Plataformas	Android compila y ejecuta; iOS Simulator ARM64 compila.
Arquitectura	Clean Architecture + MVVM con StateFlow e inyección Koin.
Funcionalidad	Registro/listado de libros y lectores; navegación; tema claro/oscuro; préstamos en construcción.
Validación	Mensajes exactos del Anexo A y reglas duplicadas como invariantes en los modelos.
Pruebas	39 pruebas automatizadas, 0 fallos.
Evidencias	Inicio claro/oscuro; cuatro fases de Libros; lector sin teléfono; Préstamos.

1.2 Tecnologías

- Kotlin Multiplatform y Compose Multiplatform para compartir dominio, presentación e interfaz.
- Coroutines y StateFlow para operaciones asíncronas y estado observable de solo lectura.
- Koin para enlazar repositorios, casos de uso y ViewModels.
- kotlin.test y coroutines-test para pruebas deterministas del dominio, datos, UI state y DI.

2. Matriz de cumplimiento

Ítem	Entregable	Estado	Evidencia técnica
1	Proyecto y rama	Cumplido	Paquete pe.edu.upeu.bibliomobil; rama feature/clean-mvvm; catálogo de versiones y dependencias.
2	Dominio	Cumplido	Libro, Lector, Prestamo, DetallePrestamo, EstadoPrestamo; invariantes y reglas propias.
3	Casos de uso	Cumplido	Registrar/Listar; errores por campo; Result<T>; resultadoDe relanza CancellationException.
4	Datos	Cumplido	Repositorios en memoria, IDs correlativos, delay de 350 ms y Mutex.
5	Libros MVVM	Cumplido	Estado exhaustivo, formulario persistente, registro protegido y cuatro fases.
6	Lectores MVVM	Cumplido	Registro/listado, validaciones, teléfono opcional y texto No registrado.
7	Koin	Cumplido	Repositorios single por interfaz, casos de uso factory, plataformas actual/expect.
8	Navegación	Cumplido	DESTINOS único, menú lateral, accesos rápidos y rememberSaveable con Saver.
9	Identidad y componentes	Cumplido	Paleta azul/teal propia, temas completos y componentes reutilizables.
10	Pruebas	Cumplido	39 pruebas, 0 fallos; supera el mínimo de 20.
II	Respuestas teóricas	Cumplido	Tres respuestas en docs/RESPUESTAS.md y salida de pruebas adjunta.

3. Arquitectura de la solución

La estructura aplica Clean Architecture con MVVM. Las dependencias del código apuntan hacia el dominio; las implementaciones concretas se conectan únicamente desde el módulo de inyección.

Capa	Responsabilidad	Elementos principales	Dependencia permitida
Presentación	Renderizar estado y delegar eventos	App, Screen, pantallas Compose, UiState, ViewModels, componentes, tema	Dominio
Dominio	Reglas e interfaces del negocio	Modelos, Repository, casos de uso, resultadoDe	Solo Kotlin/coroutines
Datos	Persistencia intercambiable	LibroRepositoryEnMemoria, LectorRepositoryEnMemoria	Interfaces de dominio
DI	Composición del grafo	dataModule, domainModule, presentationModule, platformModule	Conoce las implementaciones
Plataformas	Puntos de arranque	MainApplication/MainActivity e iOSApp/MainViewController	Shared + plataforma

3.1 Flujo de una operación

Paso	Componente	Acción
1	LibroScreen / LectorScreen	Envía cambios de campos y el evento Registrar.
2	ViewModel	Actualiza StateFlow, evita doble toque e invoca el caso de uso.
3	Registrar*UseCase	Recorta, convierte y valida; devuelve Result<T>.
4	Repository (puerto)	Define registrar/listar sin conocer la fuente de datos.
5	RepositoryEnMemoria	Protege la colección con Mutex, espera y asigna ID.
6	ViewModel + UI	Recarga, limpia el formulario y muestra éxito o error.

3.2 Regla de sustitución

Cuando se incorpore un backend REST, se reemplazarán las implementaciones de data/repository y la selección de dataModule. Los modelos, casos de uso, ViewModels y pantallas se mantienen porque consumen interfaces declaradas por dominio. Esto satisface la condición del caso: cambiar la fuente de datos sin tocar reglas ni interfaz.

4. Dominio y reglas del negocio

4.1 Libro

Campo/regla	Implementación	Mensaje exacto
Título obligatorio	No vacío tras trim	El título es obligatorio
Autor obligatorio	No vacío tras trim	El autor es obligatorio
Año obligatorio	String no vacío	El año es obligatorio
Año entero	toIntOrNull	El año debe ser un número entero
Año 1450..2026	Constantes del modelo	El año debe estar entre 1450 y 2026
Ejemplares obligatorios	String no vacío	Los ejemplares son obligatorios
Ejemplares enteros	toIntOrNull	Los ejemplares deben ser un número entero
Ejemplares no negativos	valor >= 0	Los ejemplares no pueden ser negativos

La propiedad requiereReposicion es verdadera cuando ejemplares < 3. El modelo vuelve a exigir el rango de año y la cantidad no negativa mediante requiere, evitando entidades inválidas aun si se construyen fuera del caso de uso.

4.2 Lector

Campo/regla	Implementación	Mensaje exacto
Nombre obligatorio	No vacío tras trim	El nombre es obligatorio
Correo obligatorio	No vacío tras trim	El correo es obligatorio
Formato de correo	Expresión regular del anexo	El correo no tiene un formato válido
Teléfono opcional	Vacío se transforma a null	-
Teléfono válido	6 a 9 dígitos	El teléfono debe tener entre 6 y 9 dígitos

4.3 Préstamos

Prestamo reúne un lector, una lista de DetallePrestamo y un EstadoPrestamo sellado: Solicitado, Entregado, Devuelto o Vencido(díasRetraso). El detalle acepta entre 1 y 15 días y calcula la multa a S/ 1.50 por día de retraso; por ejemplo, cuatro días producen S/ 6.00.

5. Casos de uso y datos

RegistrarLibroUseCase y RegistrarLectorUseCase reciben texto crudo para conservar la diferencia entre vacío, valor no numérico y valor fuera de rango. Cada error se devuelve por campo mediante LibroInvalidoException o LectorInvalidoException dentro de Result. Los casos Listar delegan al puerto correspondiente. El helper resultadoDe captura fallos ordinarios, pero relanza CancellationException para respetar la cancelación estructurada de coroutines.

Los repositorios en memoria asignan identificadores correlativos desde 1, preservan el orden de registro, simulan 350 ms de latencia y usan Mutex para evitar carreras. Koin los registra como single por su interfaz; así distintas pantallas y ViewModels observan la misma colección.

6. Presentación MVVM

Módulo	Estado	Comportamiento
Libros	Cargando / SinLibros / ConLibros / Error	Formulario de cuatro campos, errores individuales, conteo, tarjetas, badge y reintento.
Lectores	Cargando / SinLectores / ConLectores / Error	Nombre, correo y teléfono; presenta No registrado cuando el teléfono es null.
Inicio	Opciones generadas desde lista	Portada y accesos rápidos sincronizados con los destinos del menú.
Préstamos	Estado vacío	Módulo identificado como Préstamos en construcción.

- Las pantallas no mantienen estado de negocio con remember; reciben StateFlow desde los ViewModels.
- Los cambios de campo borran solo el error asociado, sin revalidar toda la operación.
- registrando deshabilita el botón y evita que un doble toque cree dos registros.
- Los formularios y la pantalla actual sobreviven a la recreación mediante SavedState/rememberSaveable.
- Libro.aUi() produce formatos exactos: 1998 · 1 ejemplar o 1998 · 3 ejemplares.

7. Navegación, identidad e inyección

Screen define Inicio, Libros, Lectores y Prestamos. La lista DESTINOS alimenta el menú y el título superior, evitando fuentes duplicadas. App se ejecuta dentro de KoinContext y obtiene los ViewModels con koinViewModel(). Android inicia Koin desde MainApplication, mientras iOS usa initKoinIos() desde iOSApp.swift; platformModule posee implementaciones actual vacías listas para dependencias específicas.

BiblioMobilTheme define esquemas claro y oscuro completos con una paleta azul/teal propia, distinta al verde de PharmaMobil y al morado predeterminado. ValidatedTextField, MensajeExito y EstadoVacio concentran patrones repetidos de interfaz.

8. Pruebas y verificación técnica

La suite supera el mínimo de veinte casos. Incluye fakes sin delay y configurables para fallar, reglas de modelos, validaciones exactas, comportamiento de repositorios, transición de ViewModels e integración del grafo Koin.

Grupo	Cobertura representativa
LibroTest	Título vacío, año fuera de rango y umbral de reposición con 2/3 ejemplares.
DetallePrestamoTest	Límites 0 y 16; multaPorRetraso(4) = 6.0.
RegistrarLibroUseCaseTest	Caso válido, mensajes del anexo, ID del repositorio y propagación de fallo.
RegistrarLectorUseCaseTest	Correo inválido, teléfono corto y teléfono vacío convertido a null.
RepositorioEnMemoriaTest	IDs correlativos y orden de registro.
LibroViewModelTest	Inicio vacío, formato UI, error de carga, errores de formulario, limpieza y recarga.
AppModuleTest	Resolución por interfaz, singleton y cuatro casos de uso.

8.1 Resultado ejecutado

Comando	Resultado
:shared:testAndroidHostTest	39 pruebas completadas, 0 fallos; BUILD SUCCESSFUL.
:androidApp:assembleDebug	Compilación Android satisfactoria.
:shared:compileKotlinIosSimulatorArm64	Compilación del framework iOS Simulator ARM64 satisfactoria.

Comando integral verificado:

```
.\gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug :shared:compileKotlinIosSimulatorArm64
```

9. Evidencias de ejecución

Las siguientes capturas corresponden a la aplicación ejecutada en un emulador Android. Los estados temporales usados exclusivamente para capturar Sin libros y Error fueron revertidos; el commit final conserva el repositorio funcional.

9.1 Inicio en temas claro y oscuro



Figura 1. Inicio en modo claro.

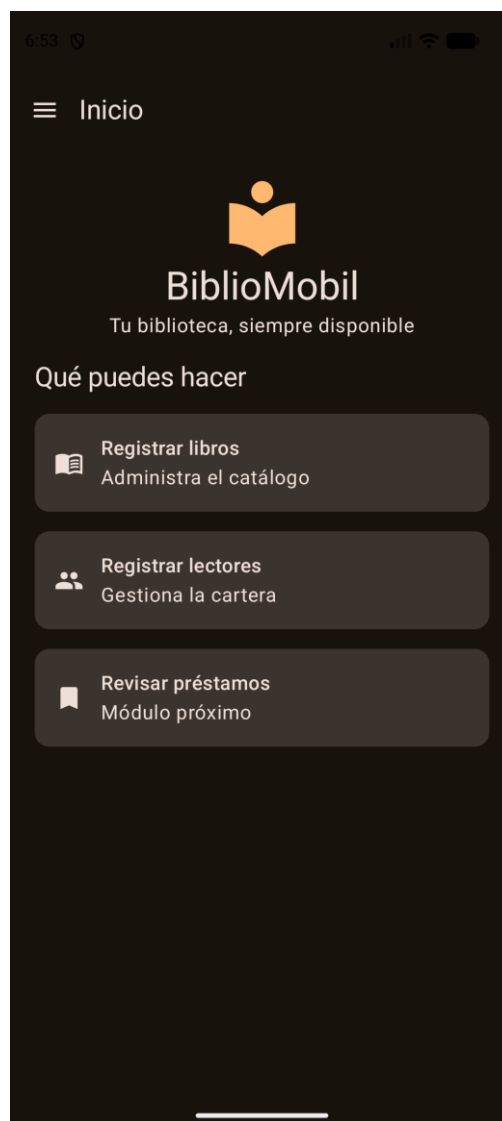


Figura 2. Inicio en modo oscuro.

Interpretación. La portada incorpora la identidad BiblioMobil, el lema y tres accesos rápidos generados desde opciones de navegación. El cambio de tema modifica los roles visuales sin perder jerarquía ni legibilidad, y el menú mantiene los cuatro destinos requeridos.

9.2 Libros: Cargando y Sin libros



Figura 3. Fase Cargando.



Figura 4. Fase Sin libros.

Interpretación. Las fases son excluyentes: durante la consulta se muestra progreso y texto informativo; al recibir una lista vacía se presenta EstadoVacio. El formulario permanece disponible y separado del resultado del catálogo.

9.3 Libros: Con libros y Error

7:31

Libros

Catálogo de libros

Título

Autor

Año Ejemplares

Registrar

1 libro

DonQuijote
Cervantes
1605 · 2 ejemplares

Pocos ejemplares

Figura 5. Fase Con libros.

7:43

Libros

Catálogo de libros

Título

Autor

Año Ejemplares

Registrar

Error

No se pudo cargar el catálogo

Reintentar

Figura 6. Fase Error con reintento.

Interpretación. El estado con datos presenta conteo, título, autor, línea secundaria con año/ejemplares y badge Pocos ejemplares cuando corresponde. El error usa el mensaje requerido y ofrece Reintentar sin mezclar el fallo de carga con los errores del formulario.

9.4 Lectores y Préstamos

7:33

≡ Lectores

Cartera de lectores

Nombre

Correo

Teléfono (opcional)

Registrar

1 lector

Ana
ana@upeu.edu.pe
No registrado

Figura 7. Lector con teléfono ausente.

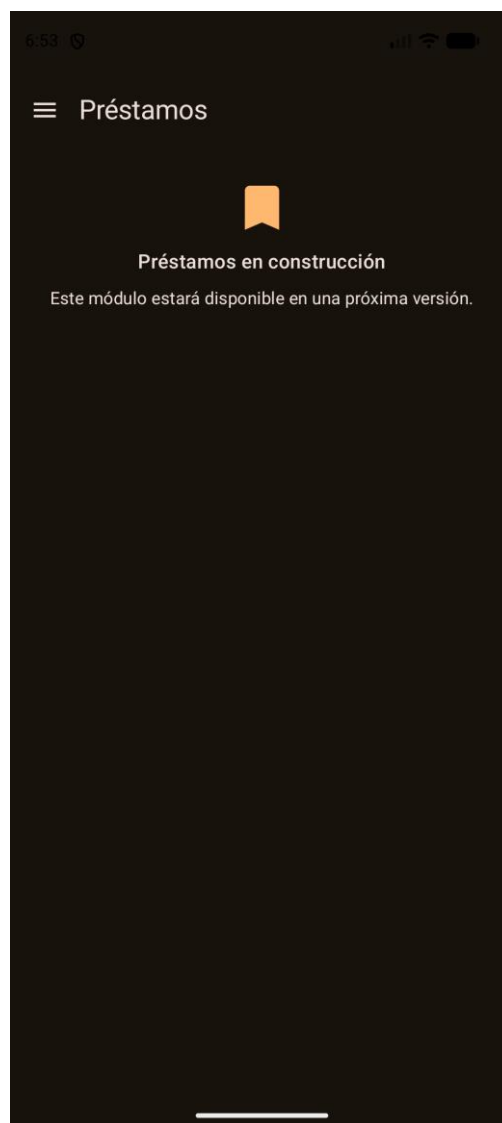


Figura 8. Préstamos en construcción.

Interpretación. El lector sin teléfono se representa exactamente como No registrado, confirmando la transformación de entrada vacía a null. Préstamos utiliza EstadoVacio con identidad de módulo, dejando claro que la funcionalidad está preparada para una versión posterior.

10. Respuestas teóricas

10.1 Migración a backend REST

Cambiarán las implementaciones de data/repository y la selección en dataModule; también podría incorporarse un cliente HTTP desde platformModule si requiere configuración específica. Permanecerán intactos los modelos, las interfaces y los casos de uso de domain, además de los ViewModels y las pantallas de presentation, porque datos implementa puertos definidos por dominio y todas las dependencias apuntan hacia esa capa.

10.2 Por qué los números ingresan como String

Si RegistrarLibroUseCase recibiera año y ejemplares como Int, se perdería la distinción entre campo vacío y texto no numérico, por lo que no podría devolver los mensajes exactos de cada caso. La pantalla o el ViewModel tendría que convertir y validar antes de invocar el caso de uso, duplicando reglas en presentación y permitiendo comportamientos diferentes entre clientes.

10.3 Repositorio single frente a factory

Con factory, cada ViewModel recibiría un repositorio en memoria diferente: un libro registrado podría desaparecer al volver al módulo o al crearse otra instancia. Con single, libros y lectores usan una fuente compartida durante la ejecución, que es el comportamiento esperado hasta incorporar persistencia real.

11. Entrega Git

Elemento	Valor
Repositorio	https://github.com/alexgm-A11/BiblioMobil-CasoBiblioteca
Rama	feature/clean-mvvm
Commit funcional verificado	5ca0ca1 - completa la entrega multiplataforma y sus evidencias
Documentación	README.md, docs/RESPUESTAS.md, docs/CAPTURAS y este informe en DOCX/PDF

12. Conclusiones

- La solución cumple los diez ítems prácticos, la parte teórica y los anexos de presentación y pruebas.
- La separación por puertos permite reemplazar la memoria por REST o persistencia local sin modificar dominio ni pantallas.
- MVVM y StateFlow centralizan el estado, preservan formularios y evitan registros duplicados.
- Las 39 pruebas y las compilaciones de Android e iOS respaldan la funcionalidad y portabilidad entregadas.
- Las evidencias visuales cubren todos los estados solicitados y el código final no contiene modificaciones temporales.

Anexo 1. Lista final de presentación

N.º	Elemento a presentar	Ubicación / evidencia	Verificado
1	Código fuente KMP completo	shared, androidApp, iosApp	Sí
2	Rama requerida	feature/clean-mvvm	Sí
3	Parte teórica	docs/RESPUESTAS.md	Sí
4	Inicio claro y oscuro	docs/CAPTURAS/inicio-*.png	Sí
5	Cuatro fases de Libros	libros-cargando/sin-libros/con-libros/error.png	Sí
6	Lector sin teléfono	lectores-telefono-ausente.png	Sí
7	Préstamos	prestamos.png	Sí
8	Salida de pruebas	39 pruebas, 0 fallos en RESPUESTAS.md	Sí
9	Compilación Android	:androidApp:assembleDebug	Sí
10	Compilación iOS	:shared:compileKotlinIosSimulatorArm64	Sí
11	Informe formal	DOCX y PDF en docs	Sí

Anexo 2. Guion breve para la sustentación

- Presentar el problema: registros dispersos y necesidad de una fuente sustituible.
- Explicar la regla de dependencia: dominio al centro; datos implementa sus interfaces.
- Demostrar Inicio, navegación, cambio de tema y persistencia de pantalla/formulario.
- Registrar un libro válido y uno inválido; señalar mensajes por campo y bloqueo de doble toque.
- Registrar un lector sin teléfono y verificar No registrado.
- Mostrar Cargando, Sin libros, Con libros y Error, incluyendo Reintentar.
- Abrir Préstamos y explicar que los modelos ya soportan estados, duración y multa.
- Cerrar con la ejecución de 39 pruebas y las compilaciones Android/iOS.

Anexo 3. Comandos de verificación

```
git switch feature/clean-mvvm
.\gradlew.bat :shared:testAndroidHostTest
.\gradlew.bat :androidApp:assembleDebug
.\gradlew.bat :shared:compileKotlinIosSimulatorArm64
```